

10. BINARY-SEARCH-TREE DATA STRUCTURE

Binary-trees and binary-search-trees

Searching

Insertion

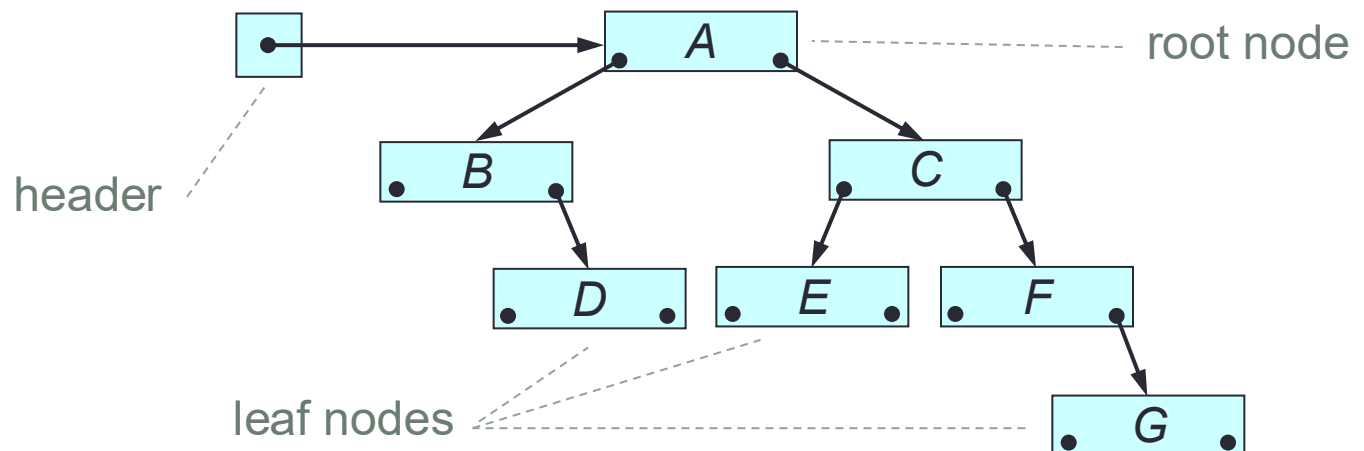
Deletion

Traversal

Implementation of sets using binary-search-trees

Binary-trees (1)

- A **binary-tree** consists of a header together with a number of nodes connected as follows:
 - Each node contains an element, plus two (possibly null) links to other nodes (its **left child** and **right child**).
 - The header contains a (possibly null) link to a unique node designated as the binary-tree's **root node**.

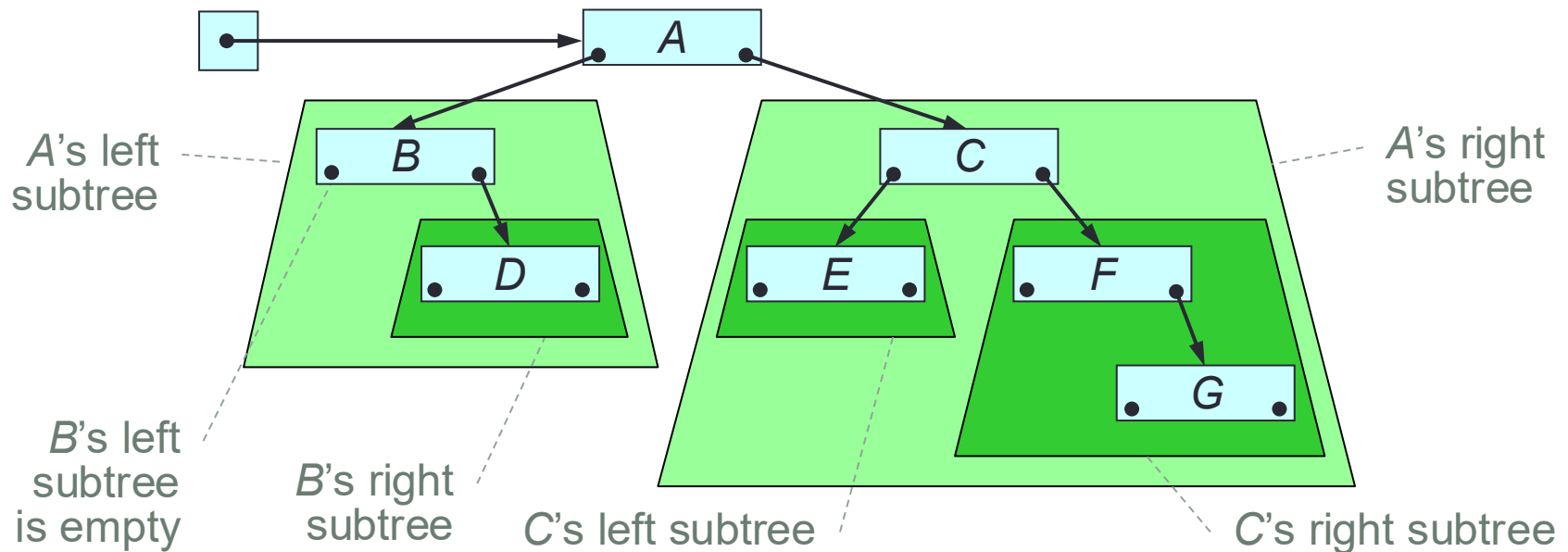


Binary-trees (2)

- A **leaf node** is one that has no children (i.e., both its links are null).
- Every node, except the root node, is the left or right child of exactly one other node (its **parent**).
- The root node has no parent; the only link to it is in the header.
- The **size** of a binary-tree is the number of nodes (elements).
- An **empty** binary-tree has no nodes. Its header contains a null link.

Binary-trees and subtrees

- Every node has a left subtree (possibly empty) and a right subtree (possibly empty). The node's **left subtree** (**right subtree**) consists of the node's left child (right child) together with that child's own children, grandchildren, etc.

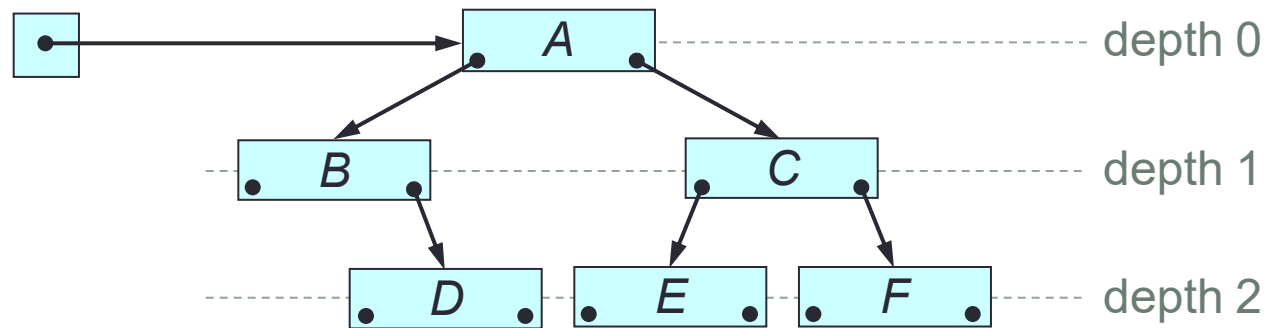


Node and tree depths (1)

- For any node N in a binary-tree, there is exactly one sequence of links between the root node and N .
- The **depth of node** N is the number of links between the root node and N .
- The **depth of a binary-tree** is the depth of its deepest node.
 - A binary-tree with a single node has depth 0.
 - By convention, an empty binary-tree has depth -1 .

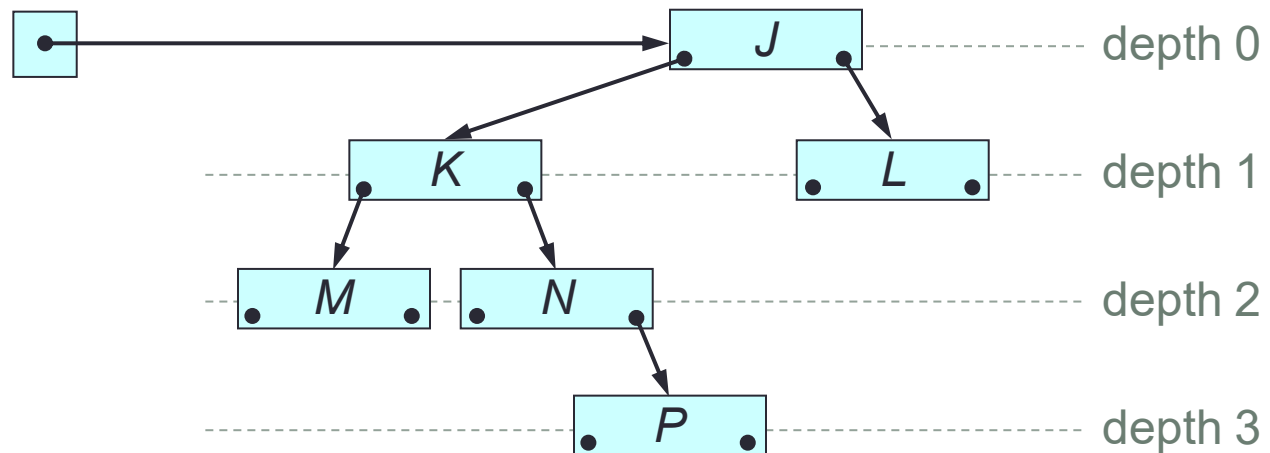
Node and tree depths (2)

- Illustration (a balanced binary-tree):



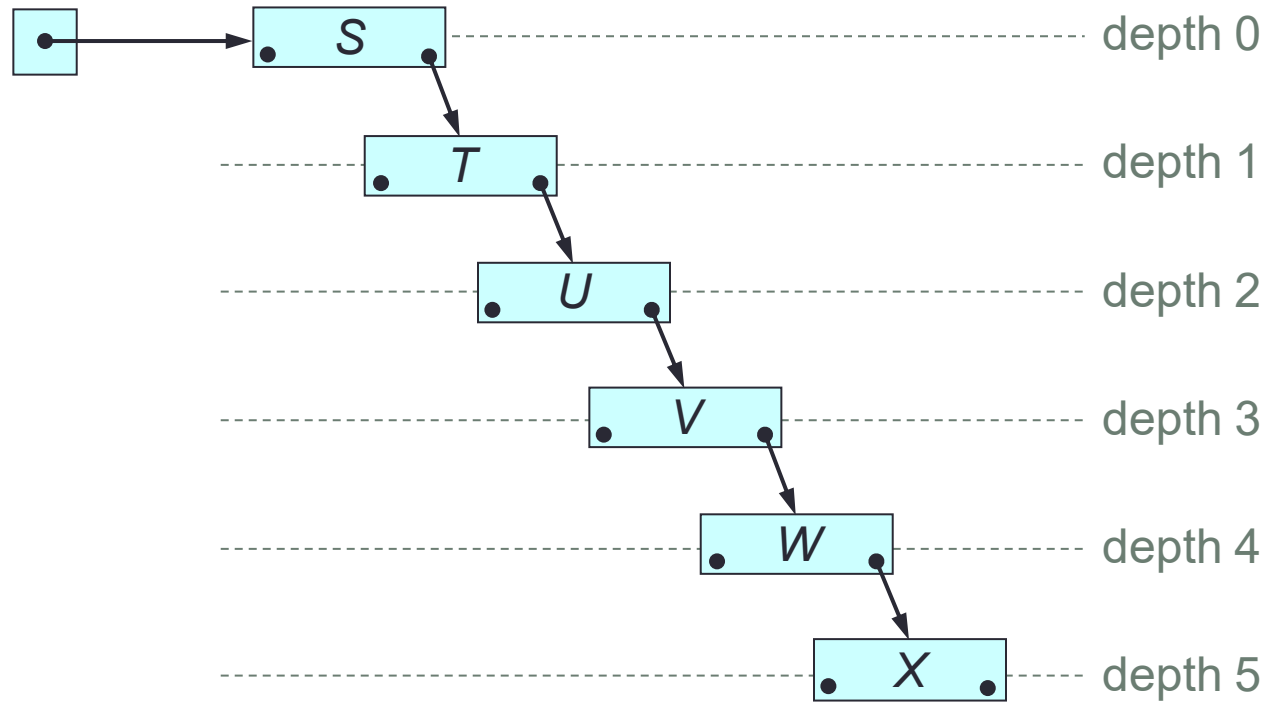
Node and tree depths (3)

- Illustration (an unbalanced binary-tree):



Node and tree depths (4)

- Illustration (an extremely unbalanced binary-tree):



Balanced binary-trees

- A binary-tree of depth d is **balanced** if all nodes at depths $0, 1, \dots, d-2$ have two children.
- This implies:
 - Nodes at depth $d-1$ may have two/one/no children.
 - Nodes at depth d have no children (by definition).
 - A binary-tree of depth 0 or 1 is always balanced.
- This is illustrated in the previous 3 slides.

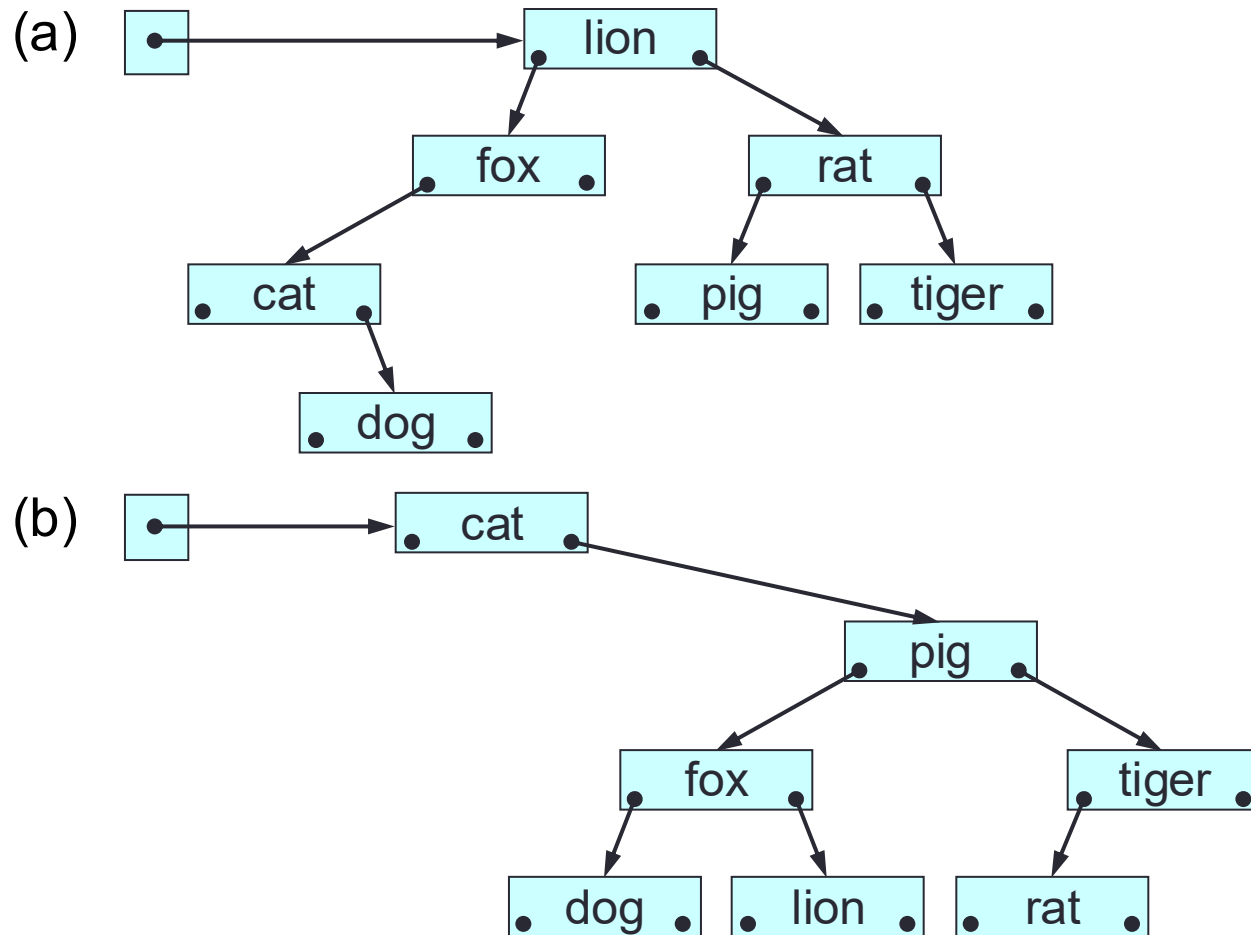
We'll see a slightly different definition when we look at AVL trees

BSTs (1)

- A **binary-search-tree** (or **BST**) is a binary-tree in which the following property holds for every node:
 - Let *elem* be the element contained in the node.
 - The node's left subtree (if non-empty) contains only elements less than *elem*.
 - The node's right subtree (if non-empty) contains only elements greater than *elem*.

BSTs (2)

- Illustrations:



BSTs (3)

- An equivalent recursive definition of BSTs is also possible.
- A **binary-search-tree** is:
 - *empty*, or
 - *non-empty*, in which case it has:
 - a root node containing an element *elem*
 - a link to a left subtree which (if non-empty) contains only elements less than *elem*
 - a link to a right subtree which (if non-empty) contains only elements greater than *elem*.

where the subtrees are themselves BSTs

BSTs (4)

- Java class implementing BSTs:

```
public class BST <E extends Comparable<E>> {  
    // Each BST object is a binary-search-tree header.  
    private Node<E> root;  
    public BST () {  
        // Construct an empty BST.  
        root = null;  
    }  
    ...  
}
```

----- BST methods (to follow)

BSTs (5)

- Java class implementing BSTs (*continued*):

```

////////// Inner class //////////
private static class Node <E extends Comparable<E>> {
    protected E element;
    protected Node<E> left, right;

    protected Node(E elem) {
        element = elem;
        left = null;   right = null;
    }
    ...
}
}

```

BST search (1)

- Problem: Search for a given target value in a BST.
- Idea: Compare the target value with the element in the root node.
 - If the target value is **equal**, the search is successful.
 - If the target value is **less**, search the left subtree.
 - If the target value is **greater**, search the right subtree.
 - If the subtree is **empty**, the search is unsuccessful.

BST search (2)

- **BST search algorithm:**

To find which if any node of a BST contains an element equal to *target*:

1. Set *curr* to the BST's root.
2. Repeat:
 - 2.1. If *curr* is null:
 - 2.1.1. Terminate yielding *none*.
 - 2.2. Else, if *target* is equal to *curr*'s element:
 - 2.2.1. Terminate yielding *curr*.
 - 2.3. Else, if *target* is less than *curr*'s element:
 - 2.3.1. Set *curr* to *curr*'s left child.
 - 2.4. Else, if *target* is greater than *curr*'s element:
 - 2.4.1. Set *curr* to *curr*'s right child.

BST search (3)

- Animation (successful search):

To find which if any node of a BST contains an element equal to *target*:

1. Set *curr* to the BST's root.

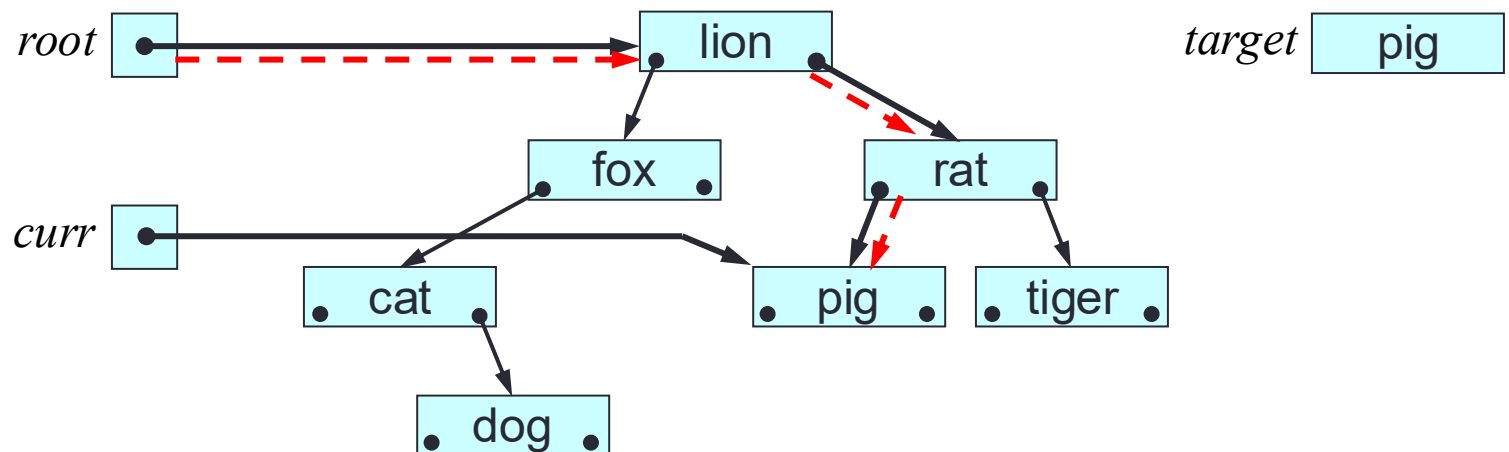
2. Repeat:

2.1. If *curr* is null, terminate yielding *none*.

2.2. **Else, if *target* is equal to *curr*'s element, terminate yielding *curr*.**

2.3. Else, if *target* is less than *curr*'s element, set *curr* to *curr*'s left child.

2.4. Else, if *target* is greater than *curr*'s element, set *curr* to *curr*'s right child.

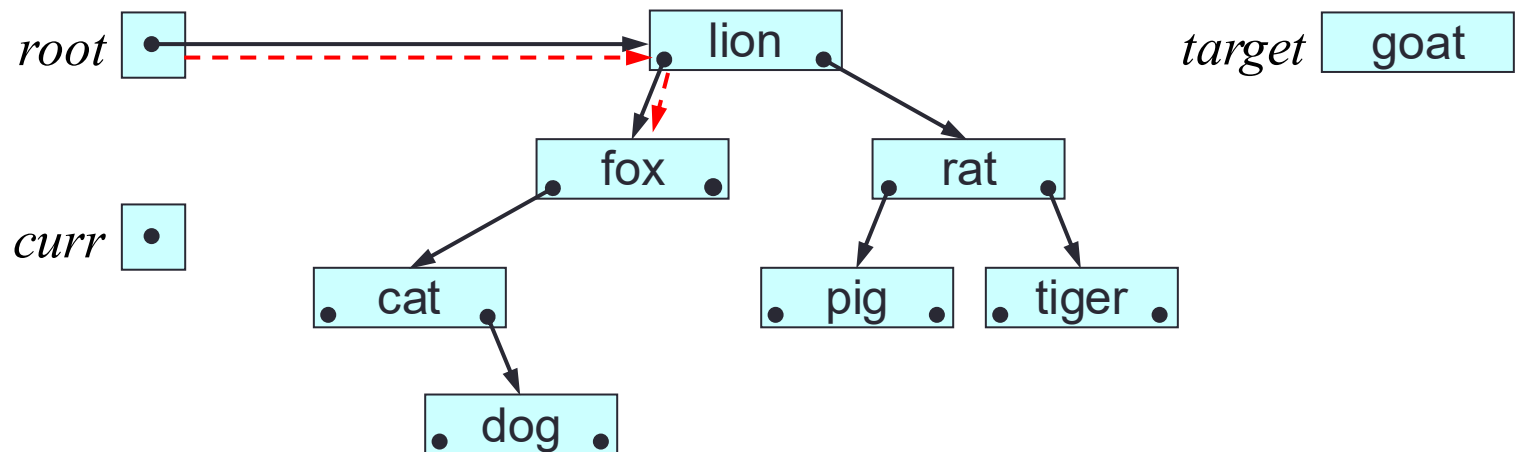


BST search (4)

- Animation (unsuccessful search):

To find which if any node of a BST contains an element equal to *target*:

1. Set *curr* to the BST's root.
2. Repeat:
 - 2.1. If *curr* is null, terminate yielding *none*.
 - 2.2. Else, if *target* is equal to *curr*'s element, terminate yielding *curr*.
 - 2.3. Else, if *target* is less than *curr*'s element, set *curr* to *curr*'s left child.
 - 2.4. Else, if *target* is greater than *curr*'s element, set *curr* to *curr*'s right child.



BST search (5)

- BST search efficiency is clearly related to the BST's *depth*. But how is its *depth* related to its *size*?
- A *balanced* BST of depth d has at least 2^d and at most $2^{d+1}-1$ nodes. Therefore:

Depth of balanced BST of size $n \approx \log n$

- An *unbalanced* BST of depth d could have as few as $d+1$ nodes. Therefore:

Max. depth of unbalanced BST of size $n = n-1$

BST search (6)

- Analysis (counting comparisons):

Let the BST's size be n .

If the BST has depth d , the max. no. of comparisons is $d+1$.

- If the BST is *balanced*, $d \approx \log n$

Worst-case time complexity is $O(\log n)$.

- If the BST is *unbalanced*, $d \approx n$:

Worst-case time complexity is $O(n)$.

BST search (7)

- Implementation as a Java method (in class BST):

```
public Node<E> search (E target) {  
    int direction = 0;  
    Node<E> curr = root;  
    for (;;) {  
        if (curr == null)    return null;  
        direction =  
            target.compareTo(curr.element);  
        if (direction == 0)  
            return curr;  
        else if (direction < 0)  
            curr = curr.left;  
        else  
            curr = curr.right;  
    }  
}
```

Including this method to give you the idea – for all other Java implementations see Moodle

BST insertion (1)

- Problem: Insert a new element into a BST.
- We must create a new node, but we must link the new node to the BST in such a way that it remains a BST.
- Idea: Proceed as if searching for that element. If the element is not already present, the search will lead to a null link. Replace that null link by a link to a leaf node containing the new element.

BST insertion (2)

- **BST insertion algorithm:**

To insert the element *elem* into a BST:

1. Set *parent* to null, and set *curr* to the BST's root.
2. Repeat:
 - 2.1. If *curr* is null:
 - 2.1.1. Replace the null link from which *curr* was taken (either the BST's root or *parent*'s left child or *parent*'s right child) by a link to a newly-created leaf node with element *elem*.
 - 2.1.2. Terminate.
 - 2.2. Else, if *elem* is equal to *curr*'s element:
 - 2.2.1. Terminate.
 - 2.3. Else, ...

BST insertion (3)

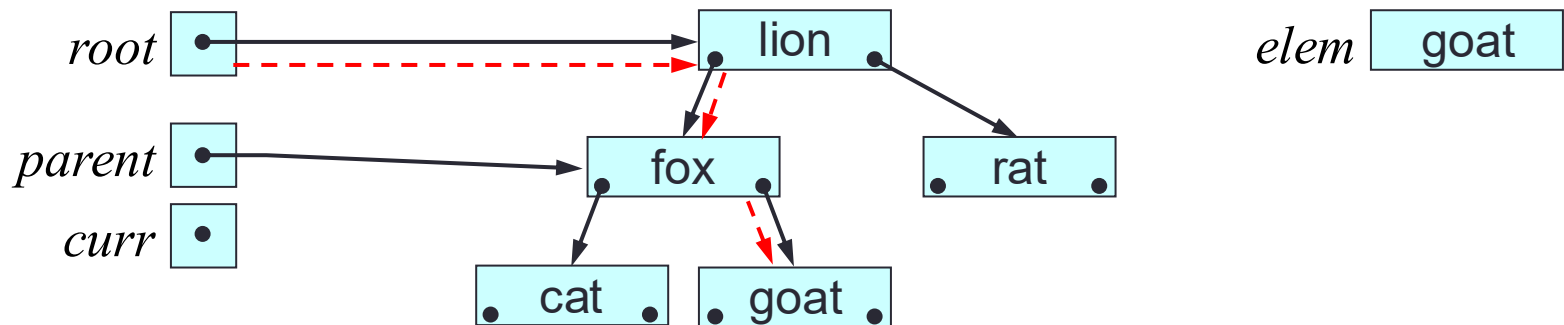
- BST insertion algorithm (*continued*):
 - 2.3. Else, if *elem* is less than *curr*'s element:
 - 2.3.1. Set *parent* to *curr*, and set *curr* to *curr*'s left child.
 - 2.4. Else, if *elem* is greater than *curr*'s element:
 - 2.4.1. Set *parent* to *curr*, and set *curr* to *curr*'s right child.

BST insertion (4)

- Animation:

To insert the element *elem* into a BST:

1. Set *parent* to null and set *curr* to the BST's root.
2. Repeat:
 - 2.1. If *curr* is null, replace the null link from which *curr* was taken by a link to a newly-created leaf node with element *elem*, and terminate.
 - 2.2. Else, if *elem* is equal to *curr*'s element, terminate.
 - 2.3. Else, if *elem* is less than *curr*'s element, set *parent* to *curr* and set *curr* to *curr*'s left child.
 - 2.4. Else, if *elem* is greater than *curr*'s element, set *parent* to *curr* and set *curr* to *curr*'s right child.



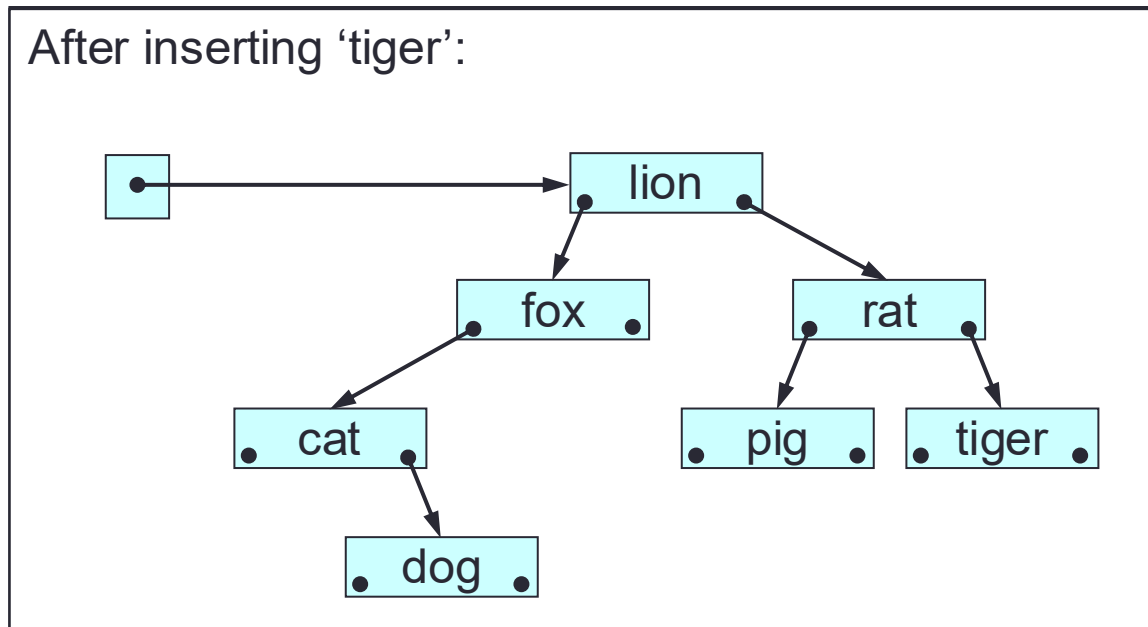
BST insertion (5)

- Analysis (counting comparisons):
No. of comparisons is the same as for BST search.
- If the BST is *balanced*:
Best-case time complexity is $O(\log n)$.
- If the BST is *unbalanced*:
Max. no. of comparisons = n
Worst-case time complexity is $O(n)$.

Example: successive insertions

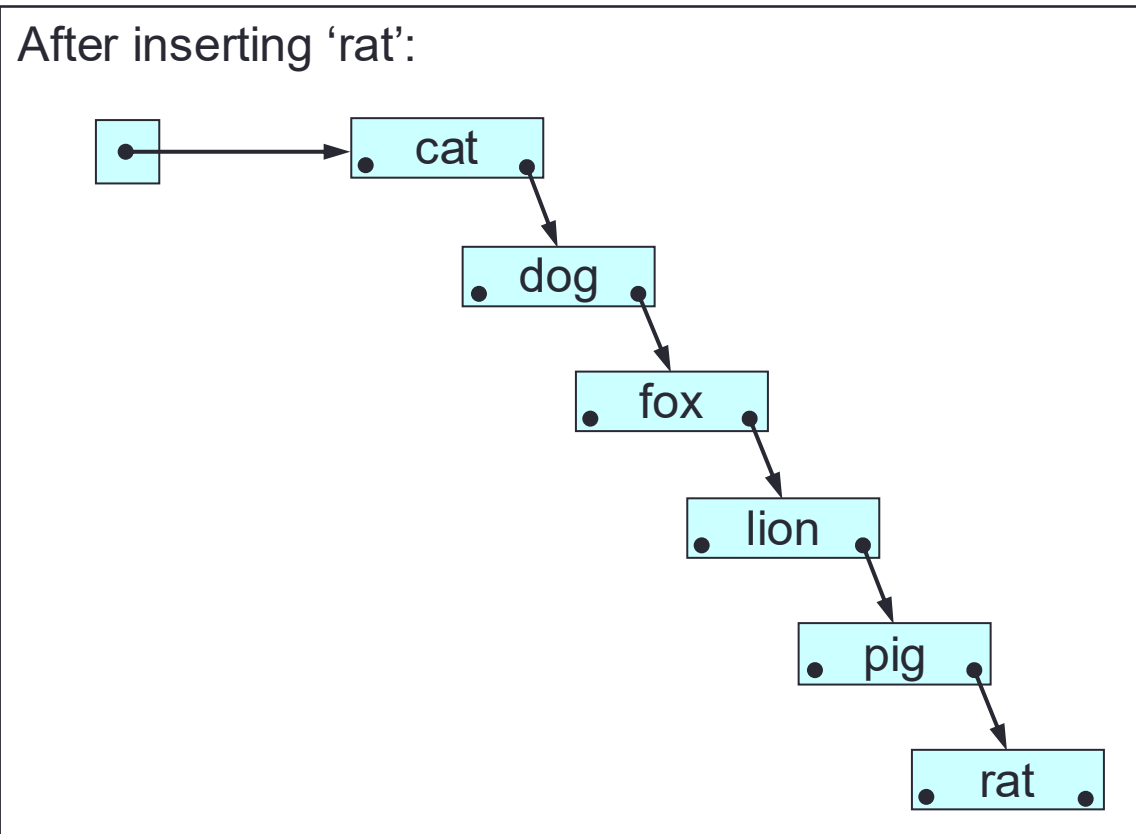
(1)

- Animation (inserting 'lion', 'fox', 'rat', 'cat', 'pig', 'dog', 'tiger' in that order):



Example: successive insertions (2)

- Animation (inserting 'cat', 'dog', 'fox', 'lion', 'pig', 'rat' in that order):



Examples

- Build a BST by inserting the values 12,7,15,9,2,6 in the given order
- Now do the same but insert the values in the order 6, 2,9, 15, 7, 12
- .. and in the order 9,15,12,6,7,2
- See demo

BST deletion

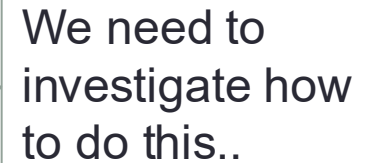
- Problem: Delete a given element from a BST.
- Idea: Search for the given element. Then delete the node containing it.
- If that node has one child (or none), deleting it is easy.
- But if that node has two children, we must take care not to leave two disconnected subtrees.

Deleting a given element (1)

- **BST deletion algorithm:**

To delete the element *elem* in a BST:

1. Set *parent* to null, and set *curr* to the BST's root node.
2. Repeat:
 - 2.1. If *curr* is null:
 - 2.1.1. Terminate.
 - 2.2. Else, if *elem* is equal to *curr*'s element:
 - 2.2.1. Delete the topmost element in the subtree whose topmost node is *curr*, and let *del* be a link to the remaining subtree.
 - 2.2.2. Replace the link to *curr* by *del*.
 - 2.2.3. Terminate.
 - 2.3. Else, ...



We need to investigate how to do this..

Deleting a given element (2)

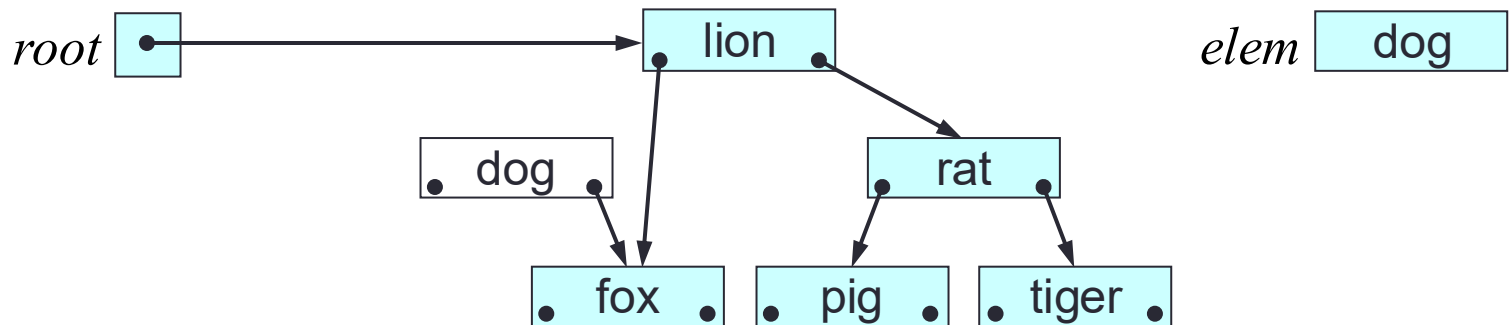
- BST deletion algorithm (*continued*):
 - 2.3. Else, if *elem* is less than *curr*'s element:
 - 2.3.1. Set *parent* to *curr*, and set *curr* to *curr*'s left child.
 - 2.4. Else, if *elem* is greater than *curr*'s element:
 - 2.4.1. Set *parent* to *curr*, and set *curr* to *curr*'s right child.

Deleting a given element (3)

- Animation:

To delete the element *elem* in a BST:

1. Set *parent* to null, and set *curr* to the BST's root node.
2. Repeat:
 - 2.1. If *curr* is null, terminate.
 - 2.2. Else, if *elem* is equal to *curr*'s element:
 - 2.2.1. Delete the topmost element in the subtree whose topmost node is *curr*, and let *del* be a link to the remaining subtree.
 - 2.2.2. Replace the link to *curr* by *del*.
 - 2.2.3. **Terminate.**
 - 2.3. Else, ...



Deleting the topmost element

(1)

- Problem: Delete the topmost element in a BST subtree.
- Four cases to consider:
 - A. The topmost node has no children.
 - B. The topmost node has a right child but no left child.
 - C. The topmost node has a left child but no right child.
 - D. The topmost node has two children.

Deleting the topmost element (2)

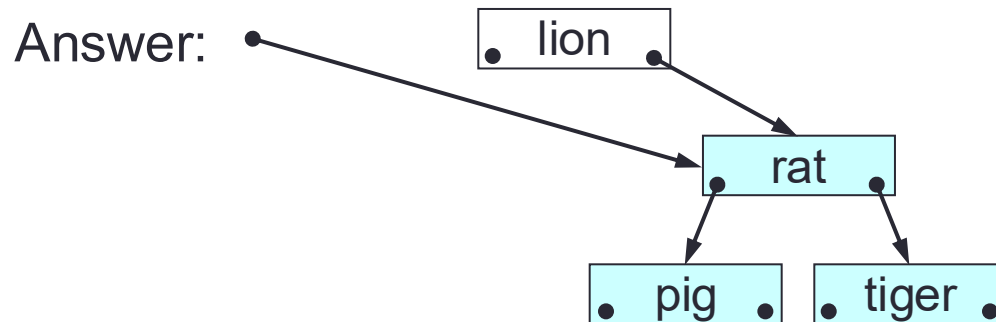
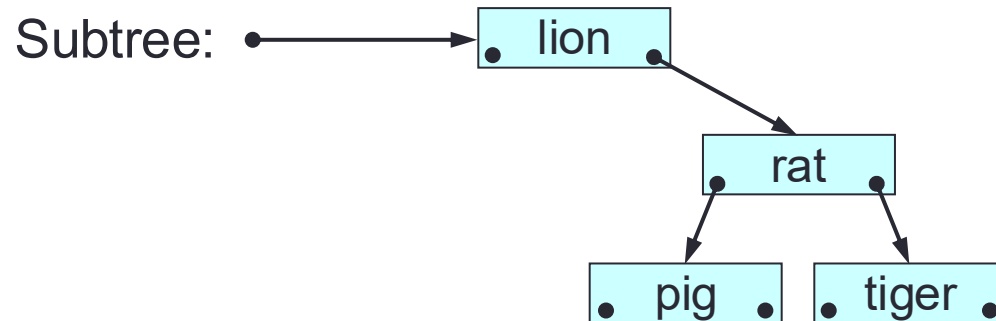
- Case A (topmost node has no children):
The answer is an empty subtree. E.g.:

Subtree: •  

Answer: • 

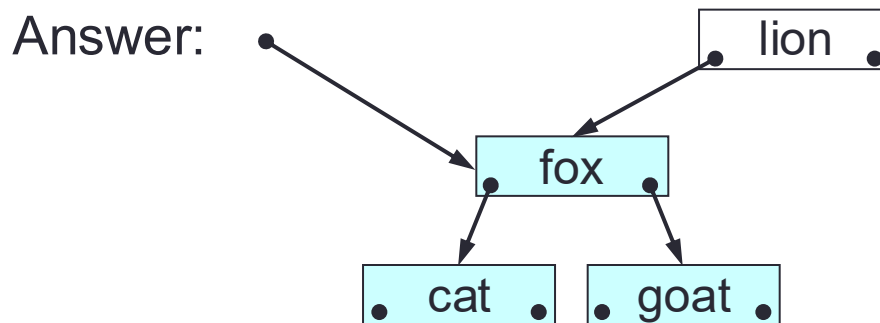
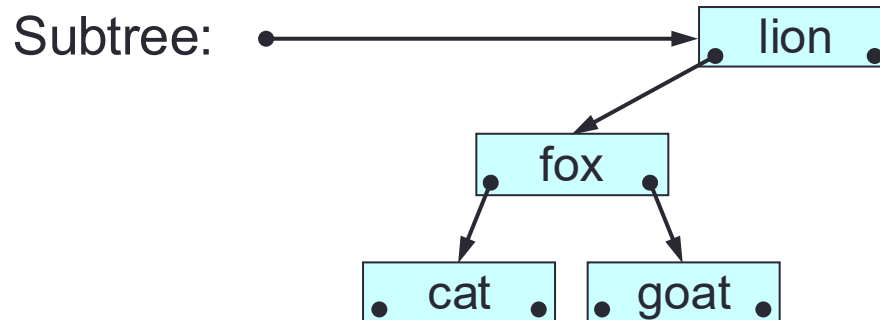
Deleting the topmost element (3)

- Case B (topmost node has a right child only):
Discard the topmost node, but retain a link to its right subtree. E.g.:



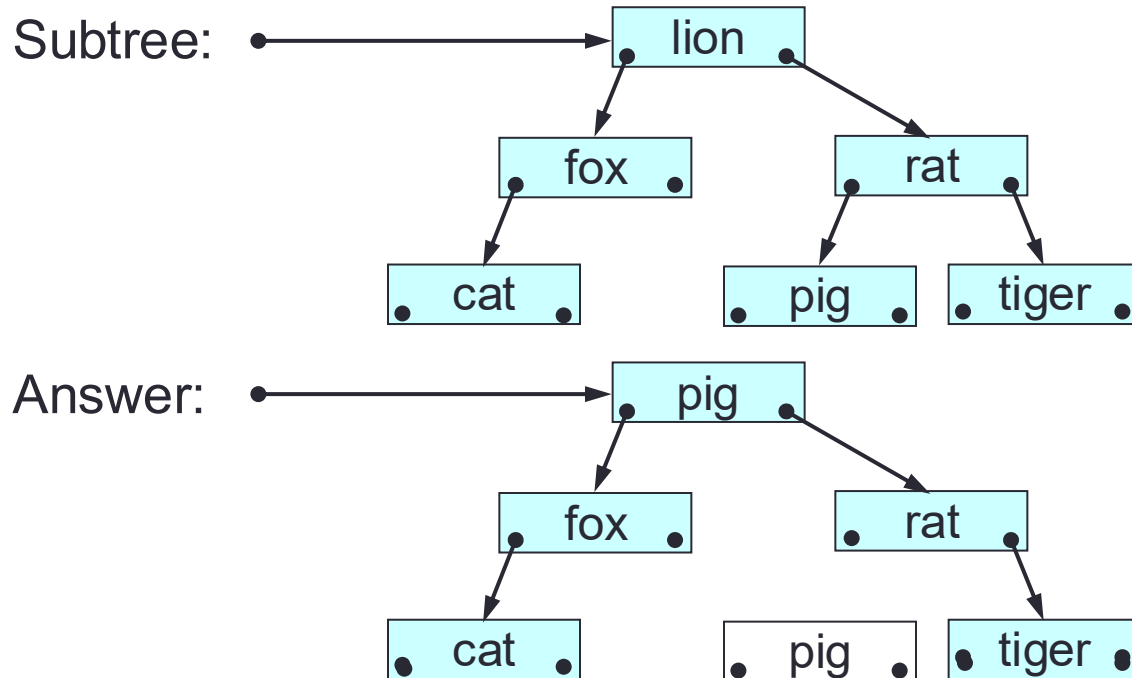
Deleting the topmost element (4)

- Case C (topmost node has a left child only):
Discard the topmost node, but retain a link to its left subtree. E.g.:



Deleting the topmost element (5)

- Case D (topmost node has two children):
Copy the right subtree's leftmost element into the topmost node, then delete the right subtree's leftmost element.
E.g.:



Deleting the topmost element (6)

- Algorithm:

To delete the topmost element in the subtree whose topmost node is *top*:

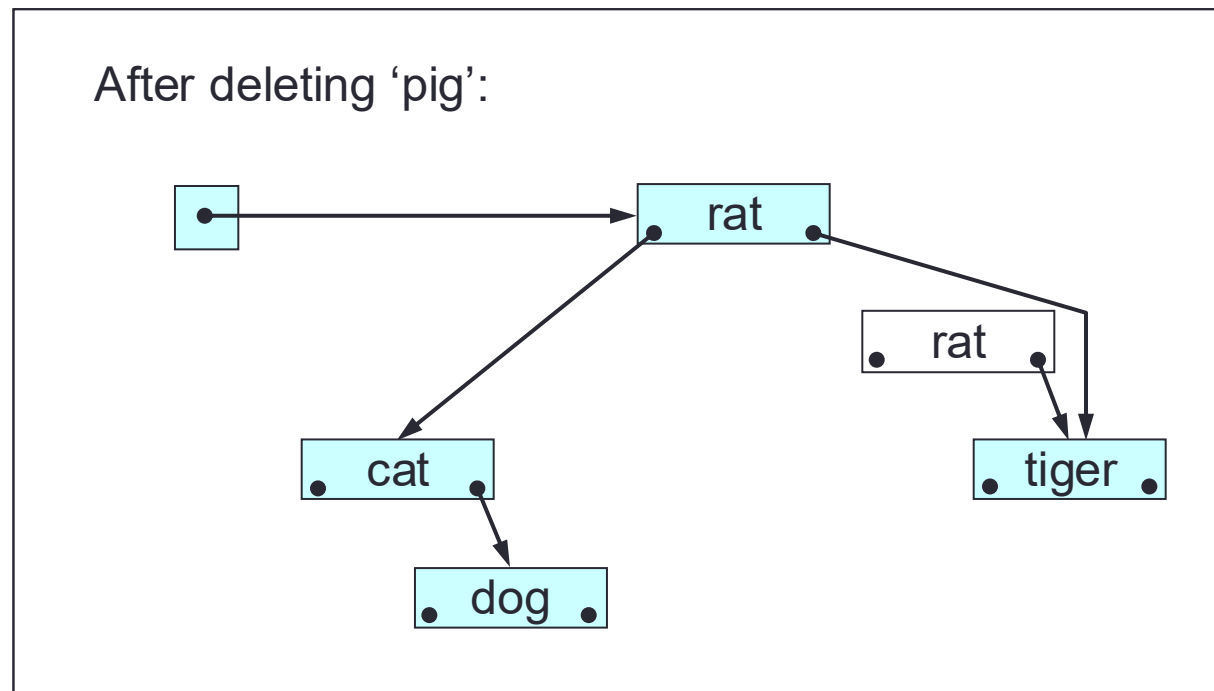
1. If *top* has no left child:
 - 1.1. Terminate with *top*'s right child as answer. ----- cases A, B
2. If *top* has no right child:
 - 2.1. Terminate with *top*'s left child as answer. ----- cases A, C
3. If *top* has two children:
 - 3.1. Set *top*'s element to the leftmost element in *top*'s right subtree. ----- case D
 - 3.2. Delete the leftmost element in *top*'s right subtree.
 - 3.3. Terminate with *top* as answer.

Deleting the topmost element (7)

- It is easy to locate the leftmost node (element) in a subtree:
 - If the subtree's topmost node has no left child, the topmost node is itself the leftmost node.
 - If the subtree's topmost node does have a left child, we follow left-child links until we find a node with no left child. That node is the leftmost node.

Example: successive deletions

- Animation (deleting 'lion', 'fox', 'pig' in that order):



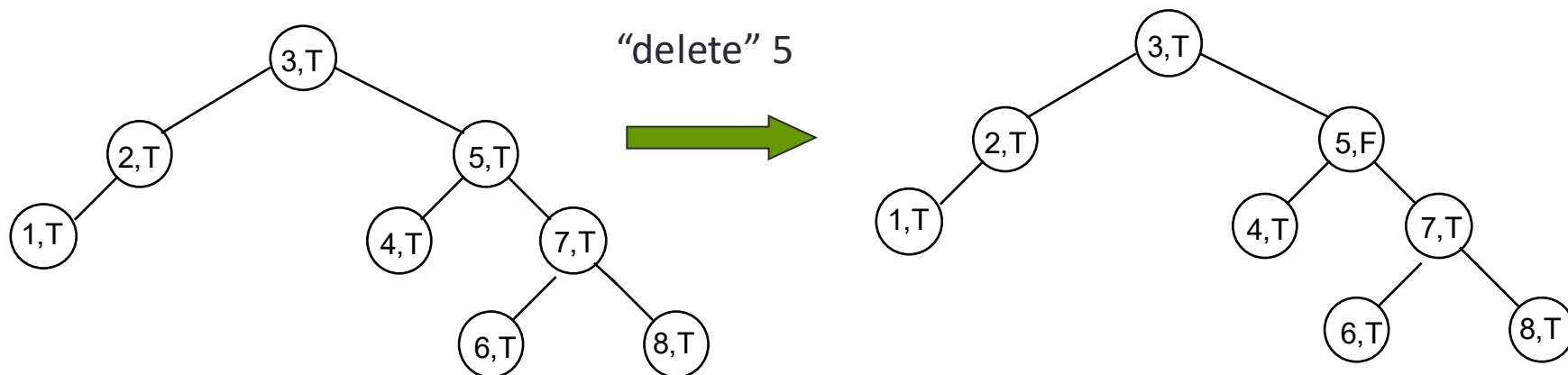
Examples

- Build BST by inserting the elements 6, 2, 9, 15, 1, 12 in the given order
- Delete the values 1, 15, 6 in the given order
- Now do it again but delete the values in the reverse order (6, 15, 1)
 - see demo

Lazy Deletion



- Rather than actually deleting nodes, just mark them as not present (by adding an extra boolean field say)
- OK if number of deletions is few
- Insertion would need to be modified too
- Reinsertion very easy
- We don't do this

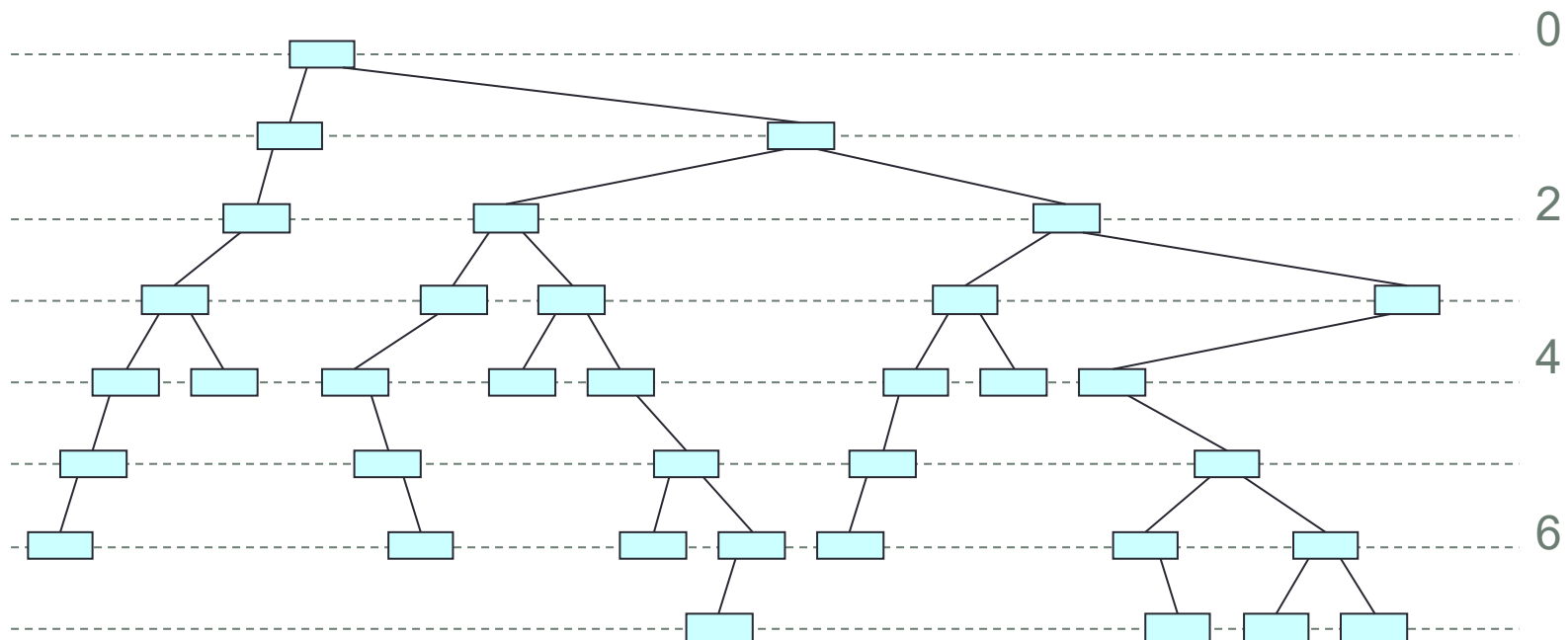


BSTs in practice (1)

- Whether a BST is balanced or unbalanced depends on the order of insertions and deletions.
- If elements are inserted in random order, the BST will probably be roughly balanced.
- If elements are inserted in ascending (or descending) order, the BST will be very unbalanced.
- Deletions can make a balanced BST unbalanced, or *vice versa*.

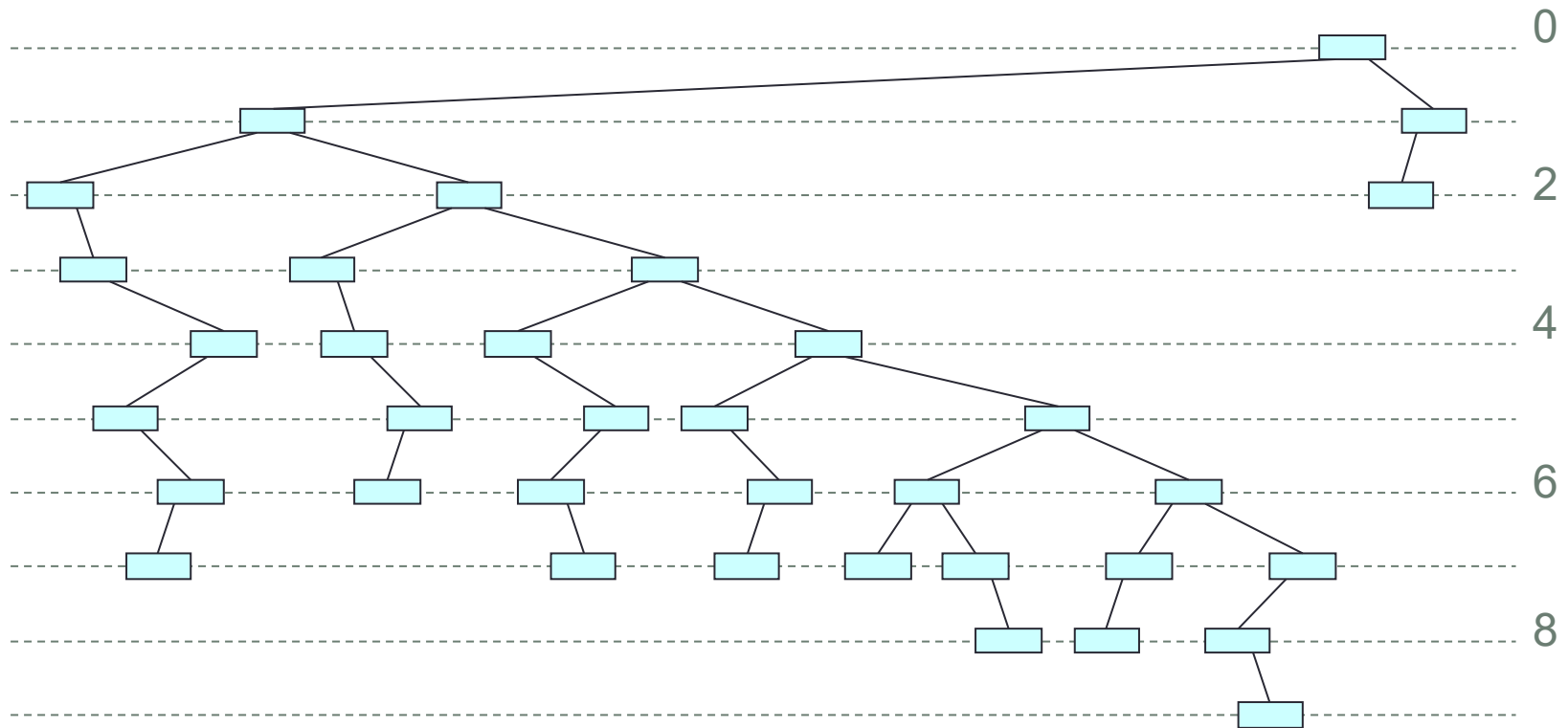
BSTs in practice (2)

- The following trials show the results of loading a BST with n randomly-generated elements.
- First trial with $n = 35$:



BSTs in practice (3)

- Second trial with $n = 35$:



Binary-tree traversal

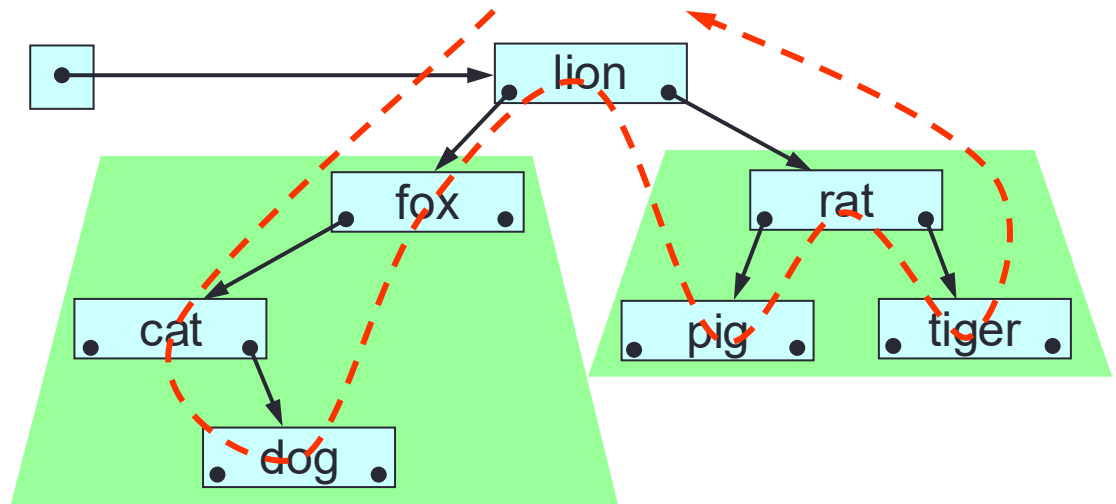
- **Binary-tree traversal:** Visit all nodes (or elements) of the binary-tree in some predetermined order.
 - **In-order traversal:** Traverse the left subtree, then visit the root node, then traverse the right subtree.
 - **Pre-order traversal:** Visit the root node, then traverse the left subtree, then traverse the right subtree.
 - **Post-order traversal:** Traverse the left subtree, then traverse the right subtree, then visit the root node.
- Note that these traversals all apply to BSTs as a special case.

In-order traversal

- **Binary-tree in-order traversal algorithm:**

To traverse, in in-order, the subtree whose topmost node is *top*:

1. If *top* is not null:
 - 1.1. Traverse, in in-order, *top*'s left subtree.
 - 1.2. Visit *top*.
 - 1.3. Traverse, in in-order, *top*'s right subtree.
2. Terminate.

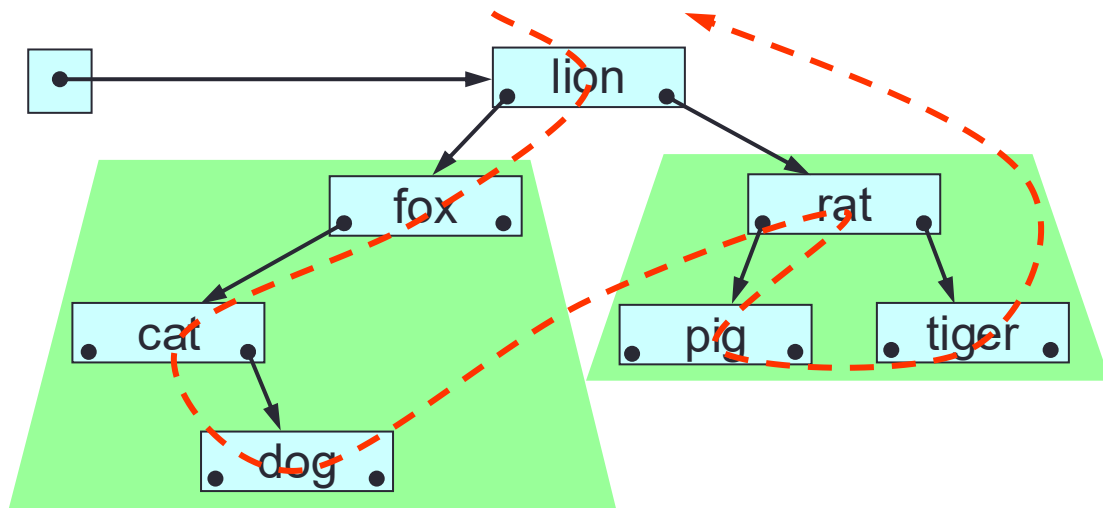


Pre-order traversal

- **Binary-tree pre-order traversal algorithm:**

To traverse, in pre-order, the subtree whose topmost node is *top*:

1. If *top* is not null:
 - 1.1. Visit *top*.
 - 1.2. Traverse, in pre-order, *top*'s left subtree.
 - 1.3. Traverse, in pre-order, *top*'s right subtree.
2. Terminate.

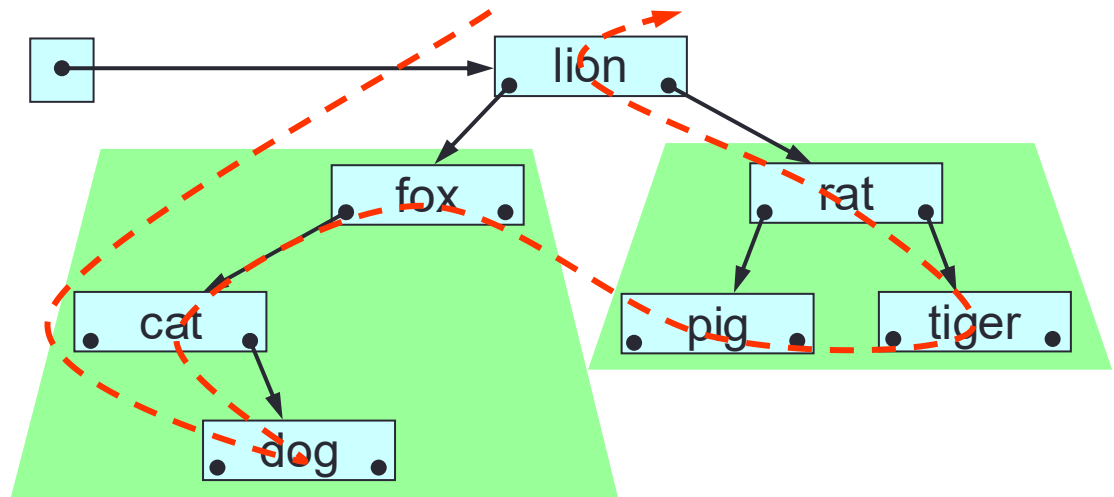


Post-order traversal

- **Binary-tree post-order traversal algorithm:**

To traverse, in post-order, the subtree whose topmost node is *top*:

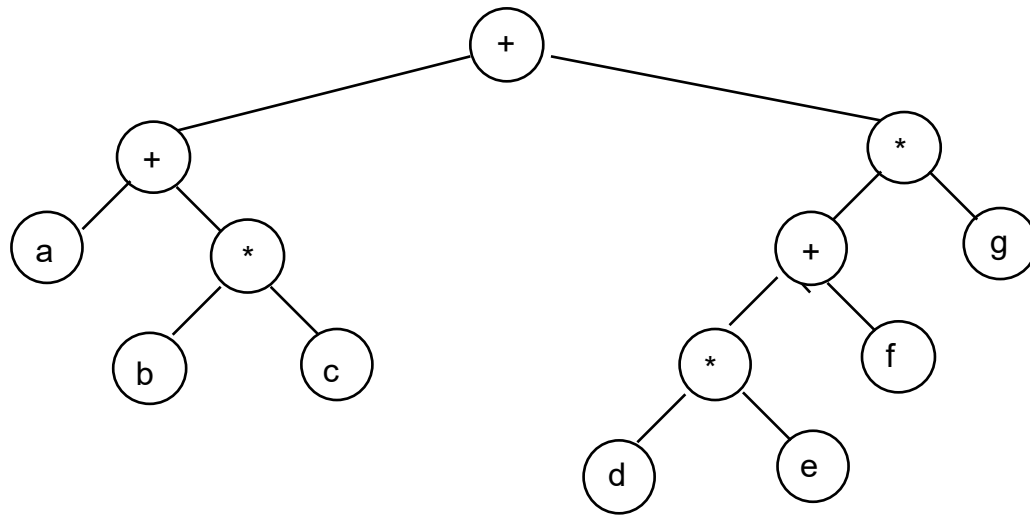
1. If *top* is not null:
 - 1.1. Traverse, in post-order, *top*'s left subtree.
 - 1.2. Traverse, in post-order, *top*'s right subtree.
 - 1.3. Visit *top*.
2. Terminate.



Use of traversals

- Inorder
 - outputs elements in ascending order,
 - infix representation of mathematical expressions
- Preorder
 - to reproduce the tree
- Postorder
 - postfix representation of mathematical expressions (reverse Polish)
 - to delete the whole tree, delete nodes in post-order.

Expression Trees

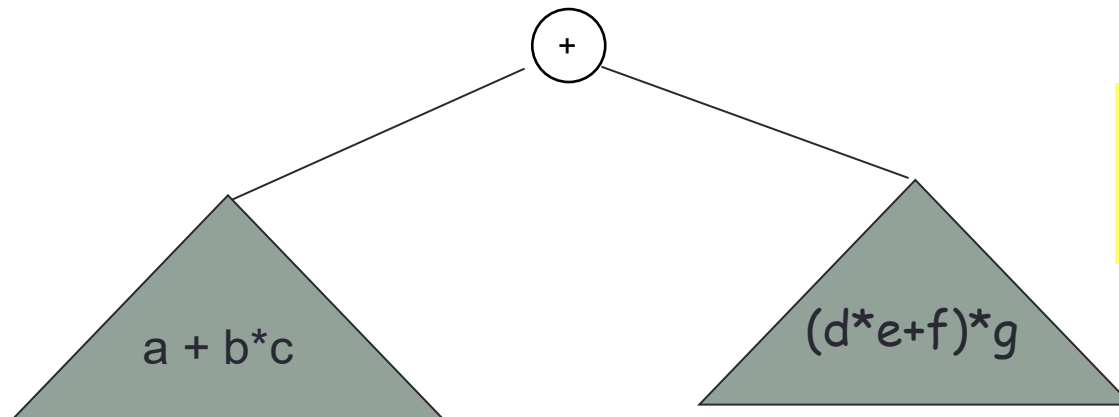


is an expression tree
for

$(a+b*c)+((d*e+f)*g)$

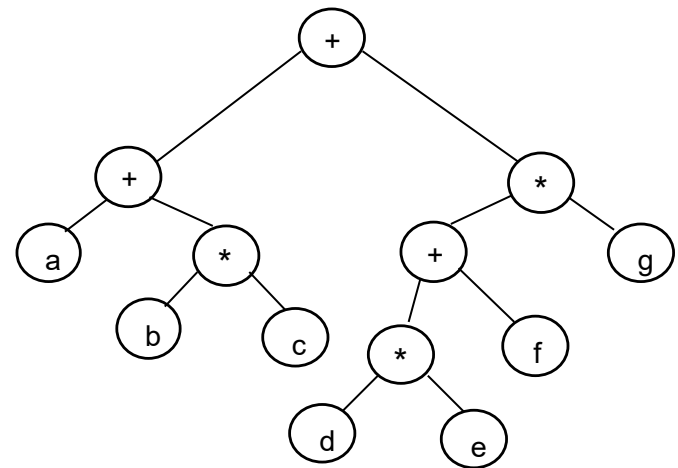
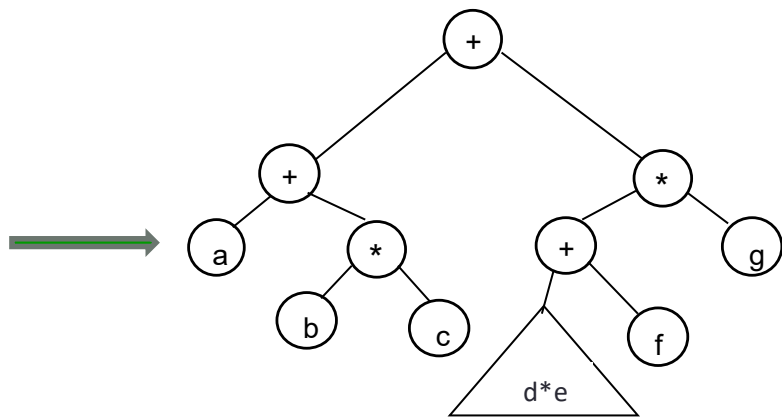
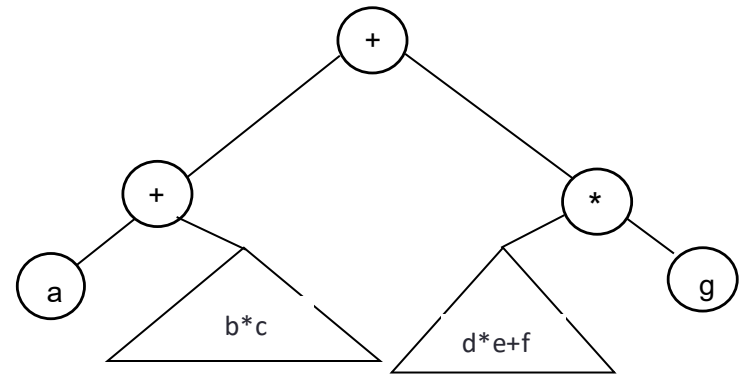
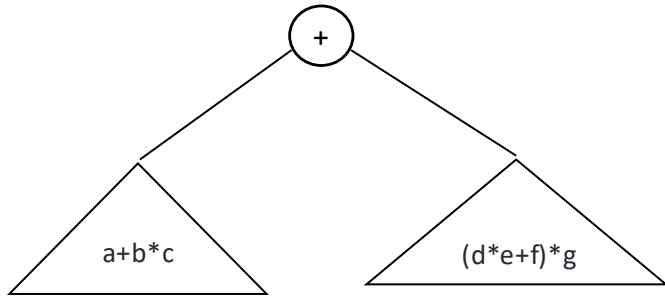
Inorder traversal gives
this expression (in infix
form)

How did we build the tree? Work inside out:

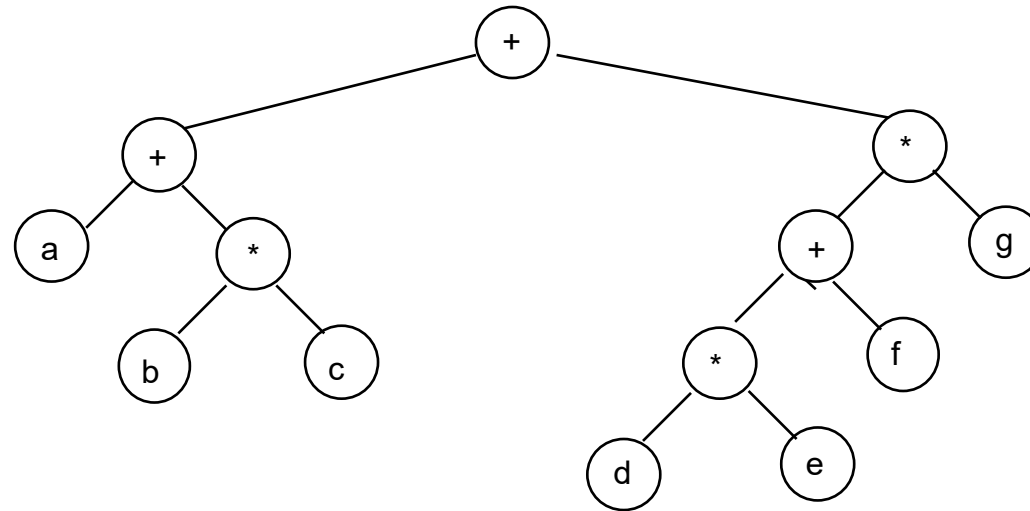


then build the subtrees:

See next slide



Expression Trees contd.



- Inorder traversal gives expression in infix form (notation)
- Postorder traversal gives the expression in postfix (or reverse Polish) form: $abc^*+de^*f+g^*+$
- Postfix notation is the form used by stack-based machines (e.g. JVM and some calculators) to evaluate expressions.

Can convert an expression in infix notation to postfix notation by creating binary tree and reading postorder transversal.

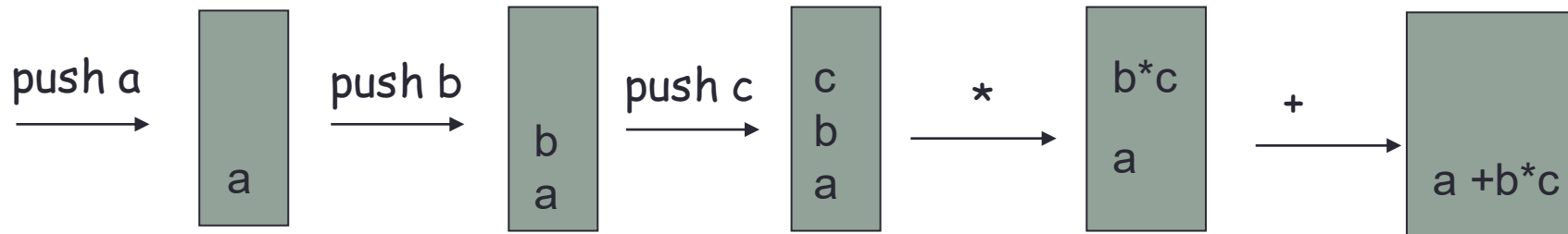
Converting an expression into list of stack commands

Take an expression in postfix form and an empty stack:

In L to R order:

- for any variable/value push to stack,
- for an operator op, pop top two elements, A and B from the stack and push result of applying operator (B op A) to stack

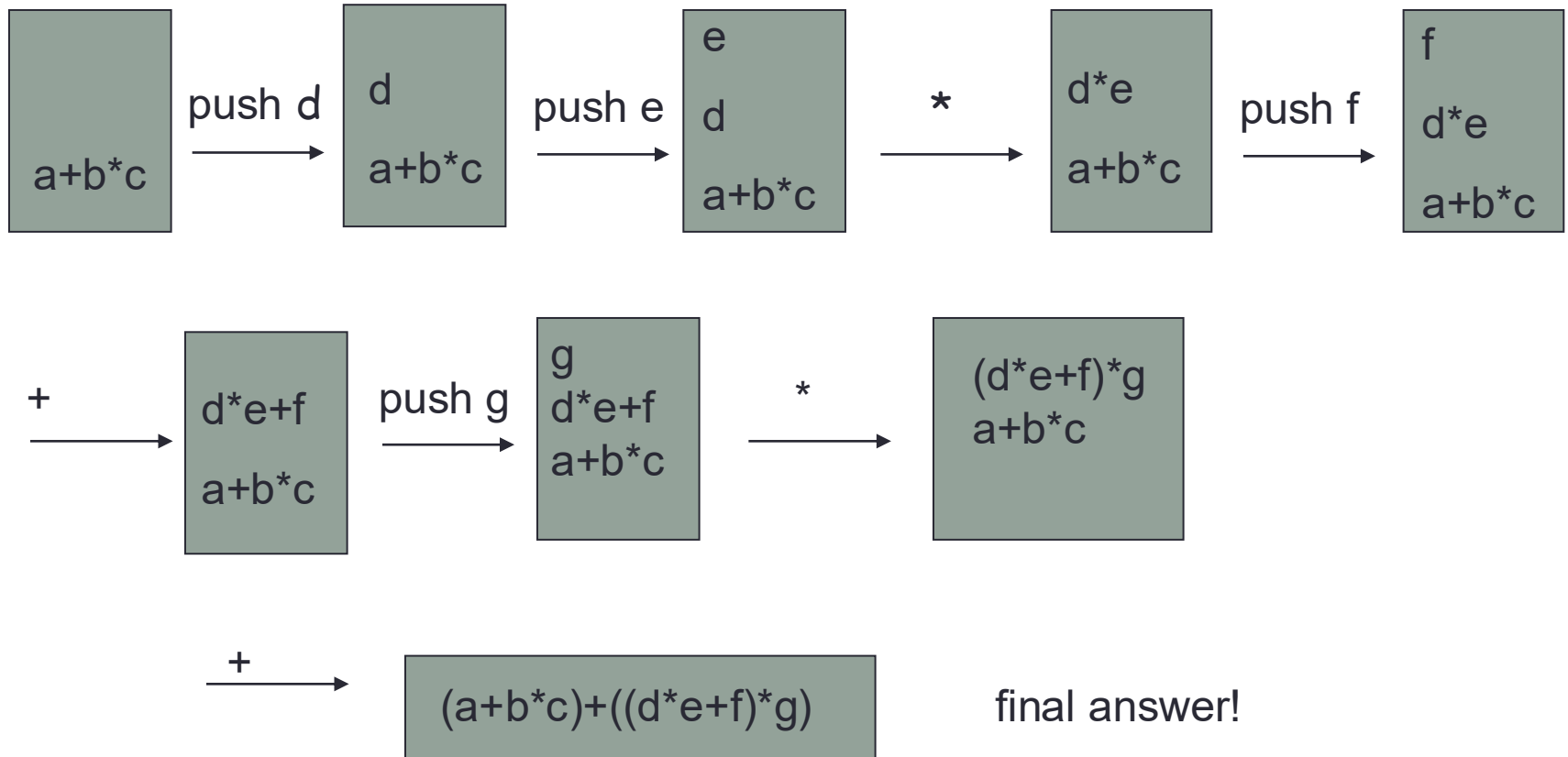
- Example: $abc^*+de*f+g^*+$



contd.

stack commands contd.

$abc^*+de^*f+g^*+$

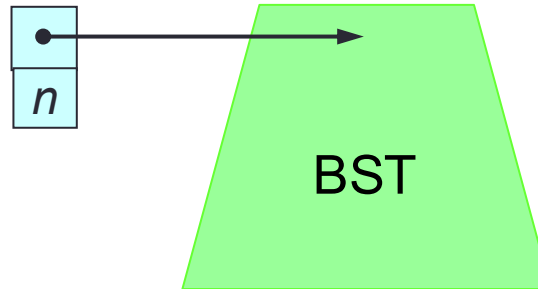


Implementation of sets using BSTs

(1)

- Represent an (unbounded) set by:
 - a variable *size*
 - a BST whose elements are the set members.

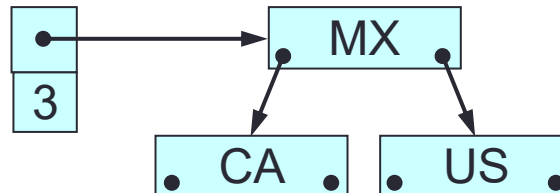
Invariant:



Empty set:



Illustration:

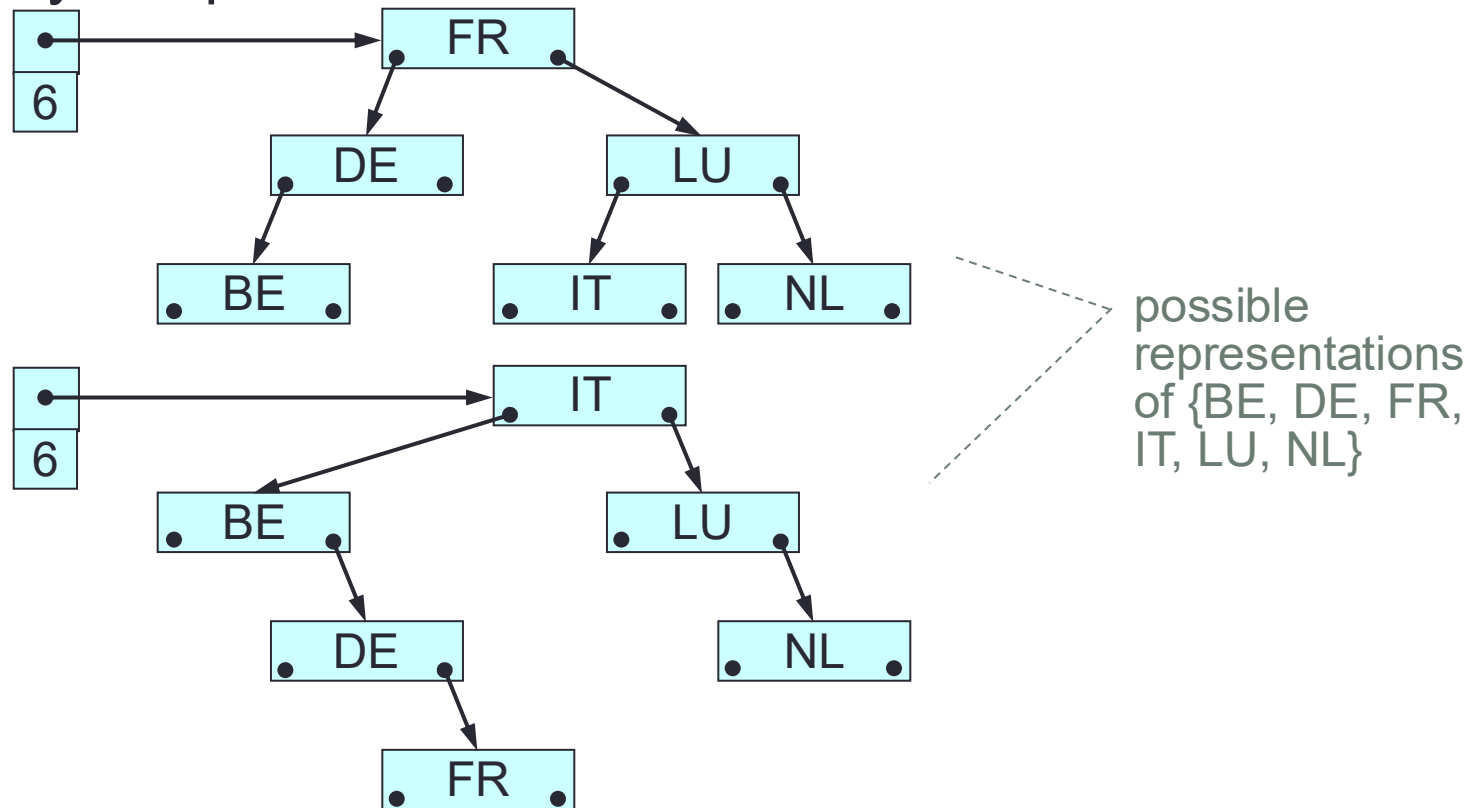


possible
representation
of {CA, MX, US}

Implementation of sets using BSTs

(2)

- Note that the BST representation of a set is not necessarily unique:



Implementation of sets using BSTs (3)

- Summary of algorithms and time complexities (NB assumes second set is also BST). Best case is when 1st BST is balanced:

Operation	Algorithm	Time complexity	
		best	worst
contains	BST search	$O(\log n)$	$O(n)$
add	BST insertion	$O(\log n)$	$O(n)$
remove	BST deletion	$O(\log n)$	$O(n)$
equals	traversal of 2 nd set combined with searches of 1 st BST	$O(n' \log n)$	$O(nn')$
containsAll	traversal of 2 nd set combined with searches of 1 st BST	$O(n' \log n)$	$O(n' n)$
addAll	traversal of 2 nd set combined with insertions in 1 st BST	$O(n' \log n)$	$O(n' n)$
removeAll	traversal of 2 nd set combined with deletions from 1 st BST	$O(n' \log n)$	$O(n' n)$
retainAll	traversal of 2 nd set combined with searches of 1 st BST	$O(n' \log n)$	$O(n' n)$

n' =
size of
set2

A note about the complexity results for set implementations

1. In Watt+Brown, the implementation of `equals`, `containsAll`, `addAll`, `removeAll`, `retainAll` methods in each case assume that the set `that` has the same implementation as `this`. All of the complexities quoted in the book make the same assumption.

E.g. in `ArraySet` implementation, in each of these methods `that` is cast to an `ArraySet` (would cause a **casting exception** if `that` were **not an** `ArraySet`). Then the `members` array can be accessed and merging is possible because we know both arrays are ordered. Hence complexity is linear (e.g. in `retainAll` complexity is $O(n+n')$).

2. In the set implementations in these lectures I still make an assumption, although not quite so serious. I assumed that `that` had an implementation whose iterator returned elements in **sequential order** (wrt a `compareTo` method). This is not always the case. (The `Set` interface says that the iterator returns all members of the set ***in no particular order***).
3. If the `Set` were implemented correctly (i.e. no assumption about `that`), the complexities would often be a lot worse.

The iterator method for the BSTSet

```
public Iterator<E> iterator() {  
    // return an iterator that visits all members of this set,  
    //in ascending order  
    return new InOrderIterator();  
}  
  
private class InOrderIterator implements Iterator<E>{  
    // An InOrderIterator object is an object will traverse  
    // in in-order, the BSTSet object representing a set  
    // iterator is represented by a stack of references to  
    // nodes still to be visited, track  
  
    //see next slide  
  
}
```

Included for interest only –
You can add to the BST class
yourself (from Moodle)

The iterator method for the BSTSet contd.

```

private class InOrderIterator implements Iterator<E>{
    private Stack<BSTNode<E>> track;
    //contains references to nodes still to be visited

    private InOrderIterator() {
        track = new LinkedStack<BSTNode<E>>();
        for (BSTNode<E> curr = root; curr!=null; curr=curr.left)
            track.push(curr);
    }

    public boolean hasNext() {
        return (!track.isEmpty());
    }

    public E next(){
        if (track.isEmpty())
            throw new NoSuchElementException();
        BSTNode<E> place = track.pop();
        for (BSTNode<E> curr = place.right; curr !=null; curr = curr.left)
            track.push(curr);
        return place.element;
    }
}

```

More advanced search-trees

- The BST's worst-case time complexity occurs when the BST is very unbalanced.
 - We can avoid this by modifying the insertion and deletion algorithms to keep the search-tree approximately balanced.
 - This leads to:
 - AVL-trees
 - red-black-trees
- see Watt & Brown +
Section 10A
- see Goodrich & Tamassia
- The library classes `java.util.TreeSet` and `java.util.TreeMap` use red-black-trees, and thus guarantee $O(\log n)$ time complexity (e.g. for insertion)..